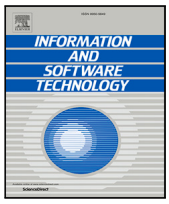
Information and Software Technology 183 (2025) 107737



Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof





Don't settle for the first! How many GitHub Copilot solutions should you check?

Julian Oertel [a,*], Jil Klünder [b,1], Regina Hebig [a]

[a] University of Rostock, Software Engineering, Albert-Einstein-Straße 22, 18057, Rostock, Germany
[b] Leibniz University Hannover, Software Engineering Group, Welfengarten 1, 30167, Hannover, Germany

ARTICLE INFO

Keywords:
Code generation
GitHub Copilot
Empirical study
Generative AI
GenAI

ABSTRACT

Context: With the integration of generative artificial intelligence (GenAI) tools such as GitHub Copilot into development processes, developers can be supported when writing code.
Objectives: As GitHub Copilot has a feature to provide up to ten solutions at once, we explore, how developers should approach those solutions with the goal of providing recommendations to achieve suitable trade-offs in finding correct solutions and checking solutions.
Methods: In this study, we analyze a total of 2025 coding problems provided by LeetCode and 17 048 solutions to solve these problems generated by GitHub Copilot in Python. We focus on three key issues: firstly, whether it is beneficial to consider multiple solutions; secondly, the impact of the position of a solution; and thirdly, the number of solutions that should be checked by a developer.
Results: Overall, our results point to the following observations: (1) solutions are not less likely to be correct if they appear at later positions; (2) when looking for a solution to a common problem, checking four to five solutions is generally enough; (3) novel or difficult problems are unlikely to be solved by GitHub Copilot; (4) skipping the first solution is advised when considering only one solution, as the first solution is less likely to be correct; and (5) checking all solutions is necessary to not miss correct solutions, but the effort is usually not justified.
Conclusion: Based on our study, we conclude that there is potential for improvement in better supporting developers. For instance, there are few cases where ten generated solutions provide more value than fewer solutions. Depending on the use scenario, it could be more useful if GitHub Copilot allowed developers to request a single, comprehensive solution.

1. Introduction

Generative artificial intelligence (GenAI) tools such as GitHub Copilot have become increasingly important for software development tasks [1]. There is a wide range of possibilities how such tools can support developers [1–3]. These use cases include supporting developers by providing code snippets or reducing their workloads by performing recurring tasks.

In order to be useful for developers one way or the other, such tools need to provide sufficiently correct solutions for their respective use case. Although this does not necessitate complete correctness for the generated code, as developers often look to GitHub Copilot for inspiration or a starting point [4], at least some level of correctness is required to provide value. However, recent research shows that the percentage of correct solutions for generated code varies strongly between different programming languages (31.28% in C to 50.0% in Python [5]), problem complexity (72.2% for easy problems to 20.42% for hard problems extracted from [6]), solution complexity [7], and whether a specific problem has been part of the training data or not (68.41% to 20.26% extracted from [6]).

While studies on the quality of generated code by GitHub Copilot have been conducted numerously, most of them do not include GitHub Copilot's ability to generate up to 10 solutions for a single programming problem. This multi-suggestion feature has the potential of increasing the amount of tasks that can be solved using GitHub Copilot. However, this assumption has neither been verified nor proven to be a double-edged sword by worsening the trade-off between the amount of checked solutions and actually finding a correct solution.

Barke et al. [4] identified two modes of developer interaction with GitHub Copilot. While interactions of developers in "acceleration"

* Corresponding author.
E-mail addresses: julian.oertel@uni-rostock.de (J. Oertel), jil.kluender@fhdw.de (J. Klünder), regina.hebig@uni-rostock.de (R. Hebig).
[1] Present address: University of Applied Sciences | FHDW Hannover, Freundallee 15, 30173 Hannover, Germany.

https://doi.org/10.1016/j.infsof.2025.107737
Received 14 September 2024; Received in revised form 13 March 2025; Accepted 26 March 2025
Available online 8 April 2025
0950-5849/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

mode aim to get immediate value without breaking their flow of work, developers in "exploration" mode are looking for options and starting points for their problem at hand. This exploration is usually slower and characterized by explicit prompting and more extensive validation of results. In this mode, developers frequently use the multi-suggestion feature of GitHub Copilot [4]. Participants of Barke et al.'s study [4] report that they validate suggested code by matching patterns against a preconceived mental model of a possible solution. While this multi-suggestion feature seems to be less frequently used in acceleration mode, we argue that the pattern matching enables developers to quickly assess larger blocks of codes. This in turn could facilitate the use of the multi-suggestion feature, lead to the correct acceptance of large blocks of code, and finally increase programming speed.

In order to support both modes, we want to investigate how a user of a generative AI tool — most likely a developer — can handle the multitude of provided solutions. Based on 17 048 solutions[2] provided by GitHub Copilot for $n = 2025$ coding problems published by LeetCode, an online platform for practicing programming for technical interviews [8], we investigate how correct solutions are distributed among up to ten provided solutions. By considering further characteristics such as the difficulty of programming problems, we aim to make recommendations about approaching the solutions in the two modes.

Outline. In Section 2 we are going to introduce GitHub Copilot and LeetCode in detail and give an overview about related works. Afterwards in Section 3, we elaborate on our methodology and study setup and present our results in Section 4. We discuss our results in Section 5 and reflect on threats to the validity of our study in Section 6. We conclude in Section 7.

2. Background and related work

In this section we are going to introduce the main components of this study and related works.

2.1. Background

AI-assisted programming has been an important research area for more than 50 years and has been gaining traction especially in recent years [9]. This assistance stretches from debugging support to automatic test case generation [10] to intelligent code completion [11]. Recently, capabilities of AI-assistance in development workflows have experienced a new surge caused by the rise of Large Language Models (LLMs) such as the GPT models [12]. Following, we describe GitHub Copilot, an AI-tool that emerged from this technology, and LeetCode, a platform for developers to practice solving coding problems.

2.1.1. GitHub Copilot

GitHub Copilot is an AI-powered tool to support developers in writing code. Solutions made by GitHub Copilot consider existing code or a natural language description of the problem or goal. It was developed by GitHub in collaboration with OpenAI and Microsoft. GitHub Copilot uses the OpenAI Codex model, which is based on the GPT-3 model, and was trained on public code from GitHub [13]. It became first available in June 2021 [14] as a technical preview version, before it was officially released in June 2022 [15]. GitHub Copilot is available as an extension for common IDEs, such as Visual Studio Code, Atom, and JetBrains IDEs. Generally, there are two ways to use GitHub Copilot. First, it can be used as an auto-completion tool, which suggests code snippets depending on the context of the current cursor position. Second, GitHub Copilot can act as a code generator, which generates code based on a prompt that can consist of a natural language description and code. The latter usually produces more complex code than the former and allows to choose from up to 10 solutions. An example of this multi-suggestion feature which is used in this study is illustrated in Fig. 1. GitHub Copilot is a commercial product and requires a subscription. For students and lecturers, GitHub offers a free education edition, which we will use in this study.

2.1.2. LeetCode

LeetCode [8] is an online platform primarily directed at candidates preparing for technical interviews. For this, the platform offers a collection of coding problems regarding general algorithmic problems and data structures in various popular programming languages. The problems are categorized by different difficulty levels and tags providing additional information either on the classification of the task (e.g., as a sorting problem) or on concepts that can be useful to solve the task such as arrays or hash maps. Additionally, the platform shows the acceptance rate of each coding problem, which is the percentage of past submissions that have been evaluated as being correct solutions. The coding problems consist of a description of the problem, one or multiple examples of input–output pairs, that may include pictures, and several constraints, regarding the input and the solution. Moreover, LeetCode provides a code snippet, that generally includes one function signature and the return type. Depending on the type of problem and the selected programming language, the code snippet can contain multiple function signatures and additional comments, e.g. that memory does not need to be freed or that a specific data structure can be used without implementing it. To solve the tasks, LeetCode provides a code editor and the option to run code against a sample of predefined test cases, or against self-written test cases by the user to get feedback before submitting the solution. After submitting an answer, LeetCode evaluates the solution against an extensive but undisclosed set of test cases and displays the result to the user. If the code passes the tests, LeetCode considers the solution to be correct. Otherwise, LeetCode displays the cause of the failure. Possible causes include wrong outputs of a test case, timeout, compilation error, or runtime error. Depending on the type of failure, the display shows the input, the expected output, a stack trace, and the number of passed test cases. In addition to a premium model, LeetCode offers a free version with fewer coding problems and reduced capabilities.

2.2. Related work

Since the release of GitHub Copilot's technical preview in June 2021 [14], numerous studies have been conducted that investigate the influence and capabilities of GitHub Copilot. Ani et al. [16] summarize in their systematic review that studies concerning GitHub Copilot include four main areas of research [16]: Developer productivity [17–22], code security [23–28], education [29–32], and code quality [5,13,22,33–36]. Additionally, several papers on practices, challenges, and limitations of GitHub Copilot have been published [2–4,9,37–40]. Apart from GitHub Copilot, studies on code quality have also been performed on similar tools such as CodeWhisperer and ChatGPT. In the following, we focus on works studying the quality of the generated code.

Research focusing on the quality of LLM-generated code has been conducted frequently in recent years. Chen et al. [13] evaluate a set of large language models using the handwritten HumanEval data set, a collection of 164 coding problems. They generate solutions in Python based on a prompt consisting of a method signature and a docstring to explain the problem and include examples. Their newly introduced Codex model, a GPT model fine-tuned on Python code from publicly available GitHub repositories, was able to solve 28.8% of coding problems on the first try. The production-oriented model used for GitHub Copilot, which we evaluate in this study, descends from this research version.

A similar study was conducted by Yetiştiren et al. [34] using the same data set and the publicly available version of GitHub Copilot. They report that GitHub Copilot was able to solve 28.7% of coding

[2] The term solution refers to an answer to a coding problem proposed by GitHub Copilot.

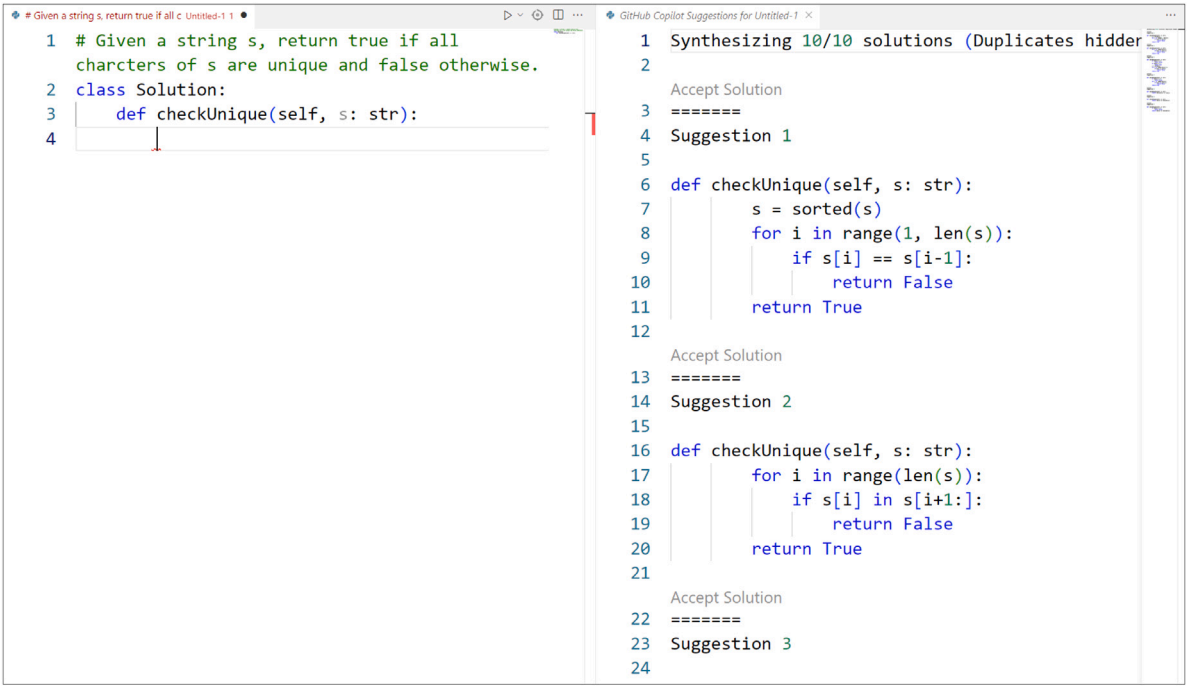


Fig. 1. Image showing how GitHub Copilot presents up to 10 solutions to a problem. The currently edited file is displayed on the left of the image, with the corresponding generated solutions presented on the right side. Accepting one of the solutions automatically fills the empty method declaration on the left.

problems. Regarding code efficiency, GitHub Copilot generated code that had the same or lower time complexity and in almost 95% of cases the same or lower space complexity in contrast to a predefined human-written solution.

In a later study Yetiştiren et al. [33] reiterate on their previous study by including Amazon CodeWhisperer and ChatGPT. Their findings include that the generated code was correct for 46.3%, 31.1%, and 65.2% of the coding problems for GitHub Copilot, CodeWhisperer and ChatGPT respectively. They attribute this change in the percentage of solved problems to the improvements of newer versions of GitHub Copilot.

Nguyen and Nadi [35] broaden the evaluation by considering multiple programming languages: Python, Java, JavaScript, and C. Moreover, they used LeetCode as data basis for the coding problems and decided on 33 problems with varying difficulties. They generated solutions to all coding problems for each programming language and used LeetCode to test the solutions for correctness. They report that 42%, 57%, 27%, and 39% of solutions were accepted for Python, Java, JavaScript, and C respectively. Additionally, they used SonarQube to analyze the generated code regarding cyclomatic complexity and cognitive complexity. However, their results show no significant differences between different programming languages.

Corso et al. [36] use GitHub Copilot, Tabnine, ChatGPT, and Google Bard to generate methods of real open source software projects based on a Javadoc description of the methods. In their study, GitHub Copilot was able to generate a correct method for 32 out of 100 descriptions and thereby outperform the other AI-assistants. However, each assistant was able to generate correct code for at least one method, that could not be generated correctly by any of the other assistants. Moreover they found, that the performance of each assistant decreased significantly when the code required dependencies outside of the current class.

Liu et al. [5] conducted a study using ChatGPT to answer 728 algorithmic problems from LeetCode in 5 different programming languages. They split their selected problems into problems created before 2021 and those created after 2021. Across languages, ChatGPT was able to solve problems created after 2021 in 20.27% of cases while solving 68.41% before. The percentage of solved problems in C++, Java, Python3, and JavaScript was similar with 44.75%, 48.74%, 50.00%, and 48.80%, respectively. However, with 31.28% it was significantly lower for C. Moreover, they further split the coding problems by their assigned difficulty rating into easy, medium, and hard coding problems, showing that more difficult problems were less likely to be solved.

To the best of our knowledge, studies on the correctness of GitHub Copilot other than the one by Chen et al. [13] only considered the first generated solution for a given problem, disregarding the possibility of correct solutions past a first incorrect solution. As GitHub Copilot can generate up to 10 solutions, this poses the question of whether it is worth it to consider more than one solution and if so, how many solutions and which solutions should be considered.

3. Research design

In the following section, we present our research goals and methodology.

3.1. Research goal and research questions

The overall goal of the research presented in this paper is twofold: First, we want to replicate results concerning GitHub Copilot's ability to generate correct solutions for coding problems. To inspect these solutions and, in particular, how they are presented, in more detail, we investigate how a developer should inspect the multitude of generated solutions for a given coding problem. Based on those goals, we pose the following research questions:

RQ1: *Is it worth it to look at more than one solution?* So far, related works have concentrated their efforts on evaluating just one solution generated by GitHub Copilot. We therefore aim to find out whether considering multiple solutions increases the number of solved problems.

RQ2: *Does GitHub Copilot increase the correctness of the first solution through specialized handling?* Generating solutions with GitHub Copilot produces up to 10 results, which are displayed in a particular order. Naturally, the first solution is of greater importance, as a developer is inclined to consider it first as well. As a measure to increase productivity, GitHub Copilot might aim to improve the quality of the first solution, for example by altering the prompt for this solution. While we cannot conclusively answer whether GitHub Copilot employs specialized handling, we expect to see significant differences in the correctness in comparison to the other solutions.

RQ3: *How many solutions should a developer look at?* Depending on the outcome of RQ1, it might be beneficial for developers to consider more than one solution to a problem to find a correct one. However, it is uncertain how many solutions need to be checked to be confident that a correct solution will be found in case it exists. Similarly, it is unclear how many solutions need to be checked until it is likely that no correct solution exists.

3.2. Data basis

To answer these research questions in detail, we consider several characteristics of the problems and their respective solutions such as the problem difficulty, inclusion in the training data of GitHub Copilot, and the position of the solutions. We therefore require a large data basis in order to increase the likelihood that each group has a sufficient sample size for statistical analysis. In line with related research [5,35], we opted for LeetCode as the basis for a multitude of different coding problems. Although these coding problems do not emerge from large industrial software projects, they have relevance for industry as they have been used in technical interviews of companies [8].

We therefore argue that those problems represent what companies expect future employees to be able to solve. Together with the metadata that LeetCode provides for every coding problem, we argue that the platform presents a suitable data basis to answer the aforementioned research questions.

3.2.1. Data source: LeetCode coding problems

LeetCode offers a catalogue of coding problems to be solved by its users [8]. In addition to the information directly needed for solving a problem (description, examples, constraints, code snippet), LeetCode includes features for finding and categorizing problems as well as exchanging thoughts and solutions about them. LeetCode assigns a difficulty rating and displays the acceptance rate of solutions that have been submitted by users for each coding problem. To discuss coding problems and their solutions, users can post and comment on each others' solutions [8].

3.2.2. Selection of coding problems

In August 2023, at the time of the study, LeetCode offered about 2900 coding problems. A part of those problems are reserved for premium subscriptions. Resulting from this, we were able to retrieve 2242 freely available coding problems at the time of the study (downloaded on August 09, 2023). To identify coding problems to be integrated in our study, we defined five exclusion criteria for LeetCode coding problems. We excluded a coding problem and removed it from our data basis as soon as one of the exclusion criteria applied. In Table 1, we present the number of excluded coding problems per exclusion criterion.

E1 *The coding problem is not meant to be solved with Python.* Most of the tasks can be solved in 18 languages. Typical languages are C, C++, Java, Python3, and JavaScript. Chen et al. [13] evaluated the ability of the Codex models to solve coding problems in Python, as this model family was fine-tuned on Python code. Subsequent studies [5,33–35] focused on Python as well or at least included Python in their set of used programming languages. For a better comparison of the results, we therefore decided to focus on Python as well.

E2 *The description contains a picture that is essential for understanding the problem.* We deemed a graphic to be essential if the information that it provides is not given in the textual description as well. Pictures are not usually a direct part of source code and cannot be handled by GitHub Copilot at the time of the study. Therefore, we considered that GitHub Copilot would not have the same input as a person solving those coding problems. To avoid an unfavorable bias against GitHub Copilot we excluded those problems from the study.

E3 *The description contains complex mathematical symbols.* We excluded matching problems for formatting reasons, as the tasks were described in HTML format. Preparing these inputs for GitHub Copilot would require a systematic reformatting effort, which we leave for future work.

E4 *The solution is expected to contain multiple functions.* Some problems require multiple functions to be implemented in order to be considered correct. For example, one coding problem expects an implementation of the data structure "stack" with typical methods such as "push", "pop", and "size". This is a pragmatic exclusion criterion that greatly eased the data collection process and was deemed acceptable by us due to the relatively low number of affected coding problems. We might consider such cases in future work.

Table 1
Overview of criteria and number of coding problems that had to be excluded.

Criterion	Description	Number of problems
–	Total number of problems	2242
E1	Non-Python	105
E2	Essential pictures	1
E3	Complex symbols	8
E4	Solution with multiple functions	103
–	Total of included problems	2025

Overall, we removed 217 coding problems (9.67%). We ended up with a total of 2025 coding problems that were further used in our study.

3.2.3. Data processing: From problems to solutions

We extracted and processed the 2025 coding problems from LeetCode using the toolchain presented in Fig. 2. We first used a Python script in conjunction with Playwright[3] to extract coding problems and metadata from the website of LeetCode. In our case, we used Playwright as a tool for automatically extracting information from LeetCode. However, this did not suffice to gather all variables. Therefore, we also used the GraphQL API that is provided by LeetCode.

Afterwards, we assembled the prompts for GitHub Copilot by merging the problem description with the given code snippet which is illustrated in Fig. 3. This created an individually prepared file for each coding problem. This step required some semi-automated work of reformatting the description to replace or delete falsely decoded symbols.

To generate the solutions, we applied GitHub Copilot to the prompts. This step was operationalized using VSCode with the GitHub Copilot extension (v1.132.0). As GitHub does not offer an API to use GitHub Copilot, we used AHK_X11 to automate this step. AHK_X11 is a Linux version of the desktop automation software AutoHotKey [42]. We then loaded a directory containing all prepared files in VSCode and

[3] Playwright is a library for end-to-end testing of web applications [41].

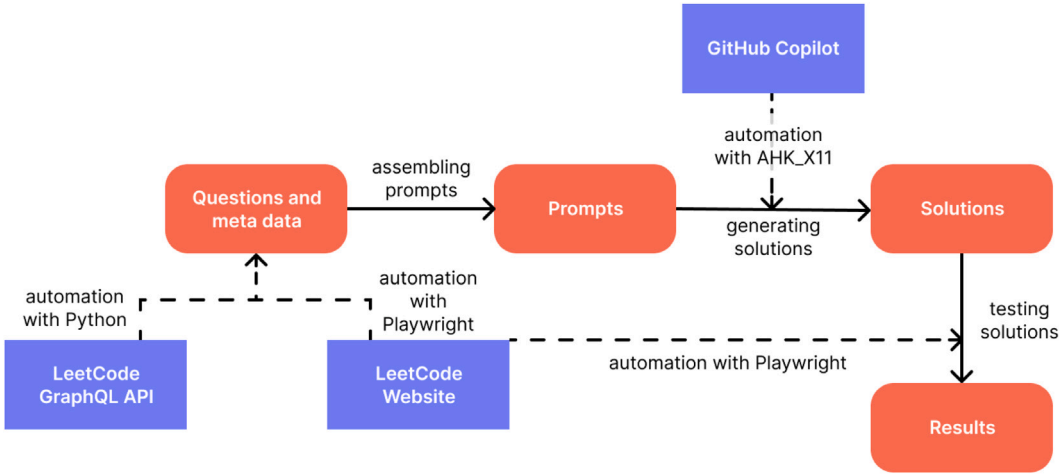


Fig. 2. Overview of toolchain used to assess GitHub Copilot's performance on LeetCode coding problems.

Given a string `s`, return true if all characters of `s` are unique and false otherwise.

Python3

```
1  class Solution:
2      def checkUnique(self, s: str):
```

(a) Separated problem description and code snippet

Python3

```
1  # Given a string s, return true if all charcters of s are unique and false otherwise.
2  class Solution:
3      def checkUnique(self, s: str):
```

(b) Content of prepared file

Fig. 3. Preparation of files for GitHub Copilot.

configured AHK_X11 to loop through each file, apply GitHub Copilot to it, and save all solutions. Usually, the code generation takes time to complete. In some cases, we observed untypical behavior during code generation, indicated by instant completion of the task without providing a result or an error. As we assumed that this behavior was caused by a rate limit, we collected these cases and reran them a few days later to get GitHub Copilot's solutions. This assumption was supported by the fact that we retrieved solutions when rerunning the tasks, which is why we consider these cases solved and integrated this data in our results.

Lastly, we used Playwright to input the solutions into LeetCode and collected the results.

3.2.4. Data cleaning

In some cases, the results provided by GitHub Copilot contained obvious errors such as duplicated solutions. These errors were manually identified and automatically removed. While GitHub Copilot would normally separate solutions with a short line of equals signs followed by the number of the solution, in some cases, this structure was compromised. Moreover, the solution itself was chaotic. For example, the method signature for

```
def searchInsert(self, nums: List[int], target: int) ->
   int
```

had changed to

```
rif searchInserthself, nums: List[int], t) // : int2 ->
  int
```

As this is not a classical syntactical error and occurred under 20 times in total, we excluded these cases from the further analyses. Nevertheless, we argue that this is an interesting observation of our study. Further investigation in the GitHub discussion on GitHub Copilot indicates that this issue has been observed by others as well.[4] The solutions generated for about 1300 coding problems of our study contained another anomaly: After an initial set of solutions, GitHub Copilot repeated a random subset of those solutions and appended them to the result set. For 95 of those coding problems, one random digit of a duplicated solution was then changed. We were not able to find any

[4] cf. https://github.com/orgs/community/discussions/48471.

Table 2
Overview of variables used for our analysis.

Variable	Name	Scale	Value range
cp_i	Coding Problem i		$i \in \{1 \dots 2025\}$
s_{k,cp_i}	Solution k of cp_i		
k	Position of solution s for cp_i	Ordinal	$k \in \{1 \dots 10\}$
$sol(cp_i)$	Solvability of cp_i	Nominal	{true, false}
$cor(s_{k,cp_i})$	Correctness of s_{k,cp_i}	Nominal	{true, false}
$a(cp_i)$	Acceptance rate of cp_i	Interval	$[0 \dots 1]$
$d(cp_i)$	Difficulty rating of cp_i	Ordinal	{easy, medium, hard}
$date(cp_i)$	Estimated date of creation	Interval	{October 25, 2013, … , August 6, 2023}
$desc-length(cp_i)$	Length of the description of cp_i	Interval	$[59 \dots 1990]$

discussion on this error. We suspect that this is a bug of the GitHub Copilot extension or in the backend of GitHub Copilot, because GitHub Copilot is supposed to remove otherwise duplicated solutions. For this reason, we opted to exclude them in our analyses as well.

3.2.5. Data set: GitHub Copilot's solutions

After having conducted the aforementioned steps, we obtained a data set of solutions offered by GitHub Copilot. Overall, we collected 17,048 solutions for 2024 out of 2025 coding problems. For one coding problem, no solution could be generated.

In cases where the solutions did not pass the LeetCode evaluation, we gathered the reported cause for the failure, and, where possible, we collected the number of test cases that were passed and the total amount of test cases available for this task. Note that in case of a compilation error, no tests were executed and thus this information was not provided.

3.3. Data availability

We assembled a replication package [43] which includes all data and scripts that were used to create the presented results. We also provide additional tables similar to those shown in Section 4.

3.4. Data analysis

In the following, we describe the steps we conducted to answer the research questions.

3.4.1. Variables used

Additionally to the coding problems themselves, LeetCode provides information for every problem that was used to further evaluate the quality of GitHub Copilot's solution. An overview of all variables used for our data analysis is presented in Table 2.

- The *position* of a solution is given by GitHub Copilot and corresponds to the order in which the solutions are shown.
- We consider a coding problem to be *solved*, if at least one of GitHub Copilot's solutions was deemed to be correct by LeetCode. Otherwise, the solvability is false.
- The *correctness of a solution* is given, if a specific solution is evaluated to be correct by LeetCode.
- The *acceptance rate* reflects the percentage of submitted solutions from all users that were deemed to be correct by LeetCode.
- The *difficulty rating* of easy, medium, or hard is assigned by LeetCode to each coding problem. However, we did not find any information on what criteria are used to make this assignment.
- We deduce an *estimated date of creation* based on the date of publicly shared solutions of users as it cannot be extracted directly. By using the GraphQL API of LeetCode we were able to retrieve the date of the oldest solution for each coding problem and use this as an estimate of the creation date. To confirm whether the date of the first solution is a good estimator, we observed LeetCode for a period of 14 days to check for newly created programming problems and track how quickly first solutions appeared. Doing so we observed that coding problems usually had at least 10 solutions within the first hour of their creation, indicating that this approach is reasonably accurate. However, as the popularity of LeetCode has likely increased over time, this method of estimating the creation date may be less accurate for older coding problems.
- The *description length* reflects the length of the natural language description of a coding problem in characters, excluding the code snippet.

We used the toolchain described in Section 3.2.3 to extract the metadata presented before. For the acceptance rate and the difficulty we used Playwright to extract the metadata directly from the website. For collecting the estimated date of creation we relied on the GraphQL API.

3.4.2. Analysis procedures for the characteristics of generated solutions

As a first step we perform a sanity check on our data, by comparing the characteristics of the solutions generated by GitHub Copilot in our study to findings from related works. For this, we compute the percentage of correct solutions and solved problems and investigate the influence of description length of the problem, creation date, difficulty and acceptance rate. We expect problems with longer problem descriptions to be less likely to be solved overall, which is in line with the observed limitations of the Codex model [13]. Liu et al. [5] observed that ChatGPT performed significantly worse on problems created after 2021 as they were likely not part of the training data used for GPT-3.5. With Codex, we do not know what problems might be included in the training data. We, therefore, aim to estimate the date up to which problems have been included by looking for a sudden decrease in the percentage of solved problems. Further Liu et al. [5] state that more difficult problems are less likely solved by GitHub Copilot. As a consequence, we should be able to see this notion as well.

3.4.3. Analysis procedures for RQ1

For the first research question, we start by investigating whether there are problems that were solved by only one solution. If such cases occur, we determine the number of solutions generated and their respective positions. Depending on the outcome, there might still exist a position on which most of the correct solutions can be found, e.g. the first one, as it might be treated differently by GitHub Copilot due to its prominent display. We therefore examine if there is a position at which most of the correct solutions to solved problems can be found. For this analysis, we split the data set into problems created before and after the date that we will determine according to Section 3.4.2, as we expect to see larger differences due to a lower amount of solutions and problems, and as problems that are unlikely to have been included in the training data are more likely to resemble typical use cases of GitHub Copilot.

3.4.4. Analysis procedures for RQ2

To answer the second research question, we first determine to what extent the position of a solution correlates with its respective correctness. In particular, we test the following null hypothesis:

$H1_0$: There is no difference in the distribution of positions for correct or incorrect solutions.

We opted for the Kruskal–Wallis test [44] to test H1. This test requires a dependent variable that is at least ordinal-scaled, which is correct for the position k (cf. Table 2). To interpret the results of the Kruskal–Wallis test, we compare the returned H value to the critical value for the χ^2 statistic that can be identified in tables for critical values for the χ^2 statistic under consideration of the degrees of freedom and the significance level [45]. We choose a significance level of $\alpha = 0.05$ for all of our tests. In case the resulting H-value of the Kruskal–Wallis test is above the identified value, we reject the null hypothesis.

To further investigate the differences in the correctness of solutions across positions, we analyzed the distributions of correct vs incorrect solutions by testing H2:

$H2_0$: There is no difference in the proportion of correct vs. incorrect solutions on position k.

We independently tested this hypothesis for each position using Pearson's χ^2-test. For each position, we compared the number of correct and incorrect solutions on this position with the sums of correct and incorrect solutions on all other positions. This way, we compared if the proportion of correct vs. incorrect solutions emerges from the same population as the proportion of the other positions. We further applied Yates' continuity correction [46] which is recommended to be used in case of small sample sizes when applying the χ^2-test to 2×2 contingency tables. For the problems created after October 2021, we observe small numbers of solved problems, which is why we apply the correction. While the correction is said to improve the accuracy of the χ^2-test in case of small sample sizes, it has no negative effect in other cases [46]. Therefore, we always present the results of the χ^2-test after having applied Yates' continuity correction.

In case the null hypothesis can be rejected for a specific position, we can conclude that there are differences in the proportion of correct vs. incorrect solutions on this position compared to the general proportion of correct vs incorrect solutions. As we tested H2 independently for each position without drawing conclusions across all positions, we did not apply the Bonferroni correction.

To further investigate the statistical differences, we compare them in detail by calculating the proportions. This reveals whether — in case of significant differences — solutions on a specific position are more or less likely to be correct than on all other positions.

In addition to the described analysis procedures, we also used a logistic regression model to analyze interactions between some of the variables. This yielded largely analogous results to the ones presented later on. However, due to a rather low goodness of fit, we do not include the results of the model in the paper.

3.4.5. Analysis procedures for RQ3

For the last research question, we adopt an exploratory approach that is based on the results for RQ1. If we find differences in the solutions on different positions, this opens up two new questions on how to approach the solutions: How many solutions should developers look at to make it likely that they do not miss correct solutions and how many incorrect solutions should developers look at before assuming that it is likely that there is no correct solution? For both questions, we imagine a developer who is eager to use one of the suggested solutions instead of rewriting the prompt or coding themselves. As a consequence, we assume a developer would want to have a 90% chance of finding a correct solution if there is one. Similarly, a developer might want to stop looking at solutions once there is an 80% chance that none of the remaining solutions is correct. Note that the choice of 90% respectively 80% is somewhat arbitrary. However, the analyses can be easily replicated for different thresholds. In both cases, we split the problems once into those created before and after the date that we identified for Section 3.4.2 and once by their assigned difficulty rating. As we expect to have greater discrepancies in those subsets, this hopefully allows us to make more nuanced recommendations.

Table 3
Overview of coding problems and solution.

Total number of problems	2025
Number of answered problems	2024
Number of solved problems	1293
Total number of solutions	17048
Number of correct solutions	5571

4. Results

We first show the characteristics of our programming problem data before we present the results of our study per research question.

4.1. Characteristics of coding problems

Overall, we collected 2025 coding problems for our study with their respective metadata as described in Section 3.4.1. Fig. 4 shows the number of coding problems by difficulty and acceptance rate. The acceptance rate is arranged into groups of ten-percent points for better visualization. In sum, the data includes 519 easy, 1046 medium, and 460 hard to solve problems. The figure also shows that easier problems tend to have a higher acceptance rate than medium and hard problems. Hard problems tend to have lower acceptance rates than easy and medium problems, correspondingly. Easy problems have an average acceptance rate of 65.68%, medium problems have an average of 49.67%, and hard problems have an average of 40.51%. Almost half of the coding problems are in the groups of 40% and 50% acceptance rate, meaning that they have acceptance rates of 40% up to 60%. Fig. 5(a) displays the distribution of coding problems based on their description length. The minimum description length is 59 characters and the maximum is 1990 characters with an average of 475.56 characters.

Fig. 5(b) shows the number of coding problems and their estimated creation dates, grouped into 8-week-intervals for better visualization. The figure shows that at the beginning of 2014 a lot of coding problems were created, which we attribute to the initial creation of LeetCode. After that the frequency of creation first dropped and then rises. Since 2020, the frequency of new problems has remained steady at 40 to 50 new problems every 8 weeks.

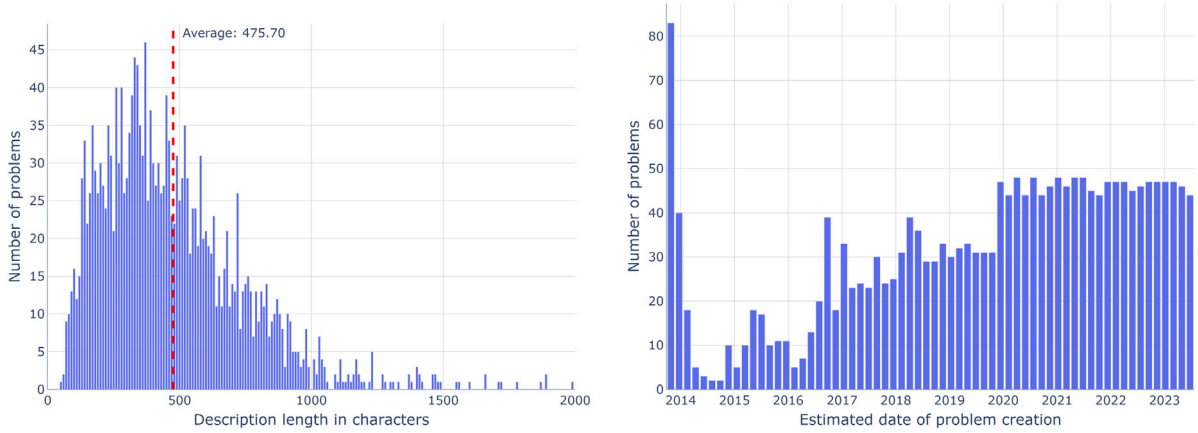
4.2. Characteristics of solutions

GitHub Copilot generated on average 8.42 solutions for each coding problem while at least one solution was generated in all cases but one (see Table 3). GitHub Copilot was able to generate at least one correct solution as determined by LeetCode, solving 63.9% (1293) of the coding problems. Out of 17 048 solutions 5571 were correct (32.67%).

Consistency with related studies: The found ability to solve problems in Python is consistent with findings from related work.

D \ AR	0%	10%	20%	30%	40%	50%	60%	70%	80%	90%	∑
Easy	0	2	3	22	71	109	122	116	74	0	519
Medium	0	2	43	152	254	278	191	92	33	1	1046
Hard	0	6	36	135	136	95	45	6	1	0	460
∑	0	10	82	309	461	482	358	214	108	1	2025

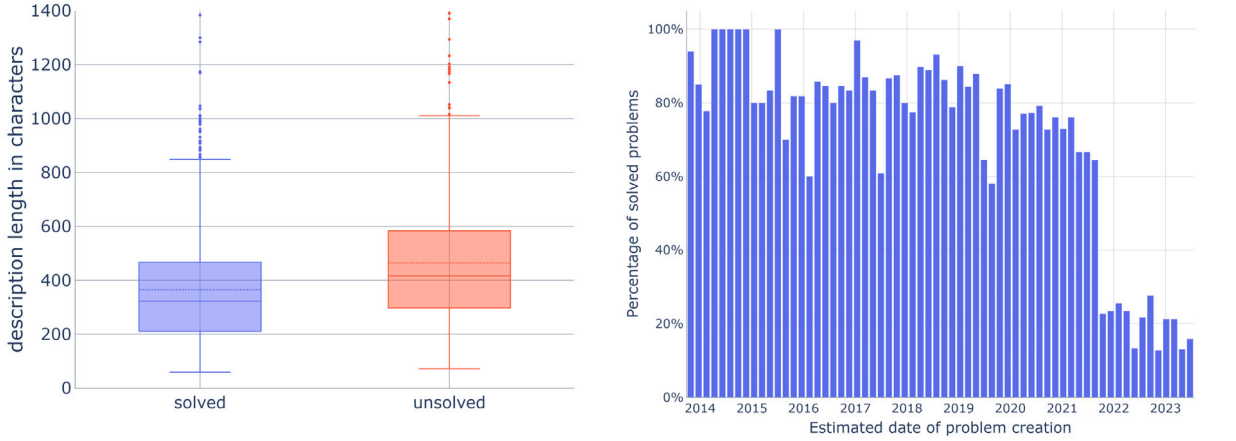
Fig. 4. Distribution of difficulty and acceptance rate of coding problems. Rows represent the difficulty of a coding problems. Columns represent the acceptance rate of user solutions to problems in ten-percentage point increments (e.g. "90%" indicates the range 90%–100%). Vertical dotted lines represent the average acceptance rates for easy, medium, and hard problems.



(a) Overview of the number of coding problems with their respective description length

(b) Number of coding problems created over time.

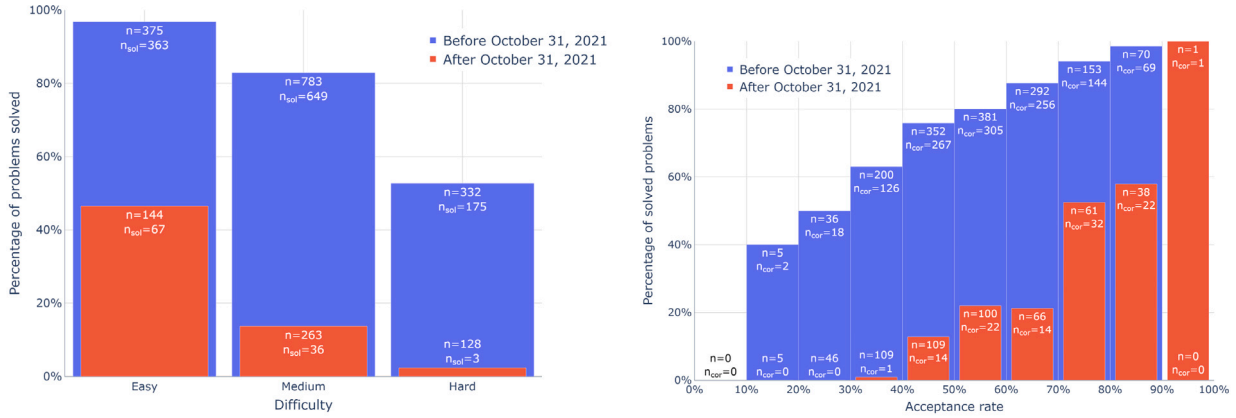
Fig. 5. Description length of coding problems and number of coding problems created over time.



(a) Box plots to illustrate the impact of the description length on GitHub Copilot's ability to solve coding problems in Python.

(b) This figure represents the percentage of solved coding problems by GitHub Copilot in correlation with their estimated date of creation.

Fig. 6. Solved vs. unsolved coding problems in relation to the description length, and the relation of creation date and the percentage of solved problems.



(a) The percentage of problems that GitHub Copilot was able to solve in relation to their assigned difficulty rating.

(b) The percentage of problems that GitHub Copilot was able to solve in relation to the acceptance rate of user submissions grouped into intervals of 10 percentage points for better visualization.

Fig. 7. GitHub Copilot's performance in relation to the difficulty rating of coding problems and the acceptance rate of user submissions. The blue columns represent problems created before October 31, 2021 and red columns presents those created afterwards. n is the number of problems and n_{sol} is the number of correctly solved problems.

As depicted in Fig. 6(a), we analyzed if GitHub Copilot's capability of solving a coding problem correlates with the length of the description of the problem.

The description of solved coding problems had an average length of 365.1 characters, while unsolved problems had an average length of 464.7 characters. The median length of problem descriptions is 323 for solved problems and 417 characters for unsolved problems. By comparing the average lengths and the median lengths, respectively, one may assume that problems are more likely solved when the task description is short.

Consistency with related studies: As described as a limitation of the Codex model [13] a longer description length occurs more often with unsolved problems in our data, too.

Fig. 6(b) displays GitHub Copilot's performance on coding problems in relation with their estimated creation date. To aggregate, we grouped the results into 8-week-intervals to calculate the percentage of problems that could be solved by GitHub Copilot, i.e. at least one correct solution was generated. Thereby, 8-week-intervals proved to be a suitable trade-off between high detail and good visualization. We discuss potential threats of validity caused by this decision in Section 6. A sharp and lasting reduction of the solved fraction of problems is visible after October 2021. For comparison, there are 1490 problems created before October 31, 2021, of which 1187 (79.6%) could be solved by GitHub Copilot while there are 535 problems created afterwards of which 106 (19.8%) could be solved. To investigate whether those results are caused by a higher proportion of more difficult coding problems, we compared the average acceptance rate and difficulty distribution of problems created before and after October 31, 2021. The average acceptance rate for problems created before October 31, 2021 is 54.5% and 51.8% for problems created afterwards. The lower acceptance rate could therefore support the notion of more difficult problems after October 31, 2021, and a lower percentage of solved problems of GitHub Copilot.

Meanwhile, the proportion of easy problems increased by 1.7 percentage points (pp) and the proportion of hard problems increased by 1.6pp, while the proportion of medium problems decreased by 3.4pp. Although we observe a slight increase in the amount of more difficult problems and a decrease in acceptance rate, this does not explain the considerable reduction in the number of solved problems after October 31, 2021.

Consistency with related studies: Our data shows the same drop in performance after 2021 as indicated by other studies [5]. Related works suggest that this performance drop is due to the likelihood that solutions to problems from before 2022 have been seen by the model during training. This aligns with our data, where the decline in performance can be attributed to neither a change in the difficulty ratings nor in the acceptance rates of the coding problems.

We further analyzed the produced results in the light of the acceptance rate and the difficulty rating. Fig. 7(a) illustrates the connection between difficulty rating and the percentage of solved coding problems.

In Fig. 7(b), we illustrate how GitHub Copilot's ability to generate at least one correct solution for problems corresponds to the acceptance rate of user submissions to the respective problem. The figure suggests a positive relationship between the acceptance rate and the percentage of solved problems.

Consistency with related studies: Liu et al. [5] show that ChatGPT's ability to solve a problem correlates with the difficulty of the coding problem. As Fig. 7 illustrates, the solutions in our data set show a similar relationship between difficulty rating and acceptance rate for solutions generated by GitHub Copilot.

Table 4
Cases of problems for which only one correct solution was generated, sorted by the position of the generated solution among all generated solutions for that problem. Generated Solutions = the number of problems for which a solution was generated on that position; Correct solutions = the number of problems that have a correct solution generated on that position; Lone correct solution = the number of problems that have a correct solution generated on that position, but on no other position; Share = Percent of lone correct solutions among the generated solutions presented at each position.

Position	Generated solutions	Correct solutions	Lone correct solution	Share
1	2024	562	25	1.24%
2	2018	682	36	1.78%
3	2002	666	37	1.85%
4	1971	651	37	1.88%
5	1919	635	23	1.20%
6	1823	593	15	0.82%
7	1697	560	17	1.00%
8	1525	508	20	1.31%
9	1254	418	22	1.75%
10	815	296	7	0.86%
All	2024	1293	239	11.80%

In summary, the characteristics of the solutions generated by GitHub Copilot in our study are consistent with insights about generated solutions in related studies. Given this successful sanity check, we continue to address our research question in the remainder of this section.

4.3. RQ1: Is it worth it to look at more than one solution?

Of the 1293 problems that were solved by at least one of the generated solutions, 239 problems (18.48%) received exactly one correct solution — also in many cases where GitHub Copilot generated multiple solutions. Table 4 summarizes these cases. Furthermore, the table shows that we find correct solutions on each position, also in cases where there is no other correct solution ("lone correct solutions"). Between 0.86% and 1.88% of the generated solutions for each position are such lone correct solutions. This is the first indication that there is no position in the list of generated solutions that is guaranteed to hold a correct solution, in case one exists. Note that this is also true when splitting the data set in problems created before October 31, 2021 (in sum, 15.04% of problems have lone correct solutions) and problems created after (9.84%).

However, for most of the 1293 problems that have a correct solution, multiple correct solutions have been generated. Thus, is it still possible to find correct solutions to most of the solved problems when looking at one position only? Here we found that the position with the most correct solutions has correct solutions for 682 problems, which is only about 53% of the solved problems (see Table 4). Interestingly, this is not the first position, but the second. Indeed, all positions from 2 to 6 provide more correct solutions than the first position.

Even when splitting up the data based on the creation date of the problem (cf. Table 5), we observe the same trend. For problems published before October 2021 the position with the most correct solutions (Pos. 2) has correct solutions for 646 problems, while overall 1187 of these problems have been solved (54%). For problems published after October 2021 the position with the most correct solutions (Pos. 6) has correct solutions for 41 problems, while overall 106 of these problems have been solved (38%).

Is it worth it to look at more than one solution? It is not the best strategy to consider only a single solution generated by GitHub Copilot for a problem. Depending on the positions considered and under the assumption that the same position is considered every time, a developer will miss correct solutions for nearly half of the problems that were solved by GitHub Copilot. For problems created after October 2021, even more than half of the correct solutions would be missed.

4.4. RQ2: Does GitHub Copilot increase the correctness of the first solution through specialized handling?

Based on the results from RQ1, one may assume that a correct solutions is, to some degree, randomly hidden among all solutions. We therefore analyzed the distribution of correct solutions for the coding problems. We found that in 43.5% of the observed cases where GitHub Copilot provided a correct solution, already the first solution was correct. To further investigate the distribution of correct solutions across positions, we tested H1 as described in Section 3. We have 9 degrees of freedom (emerging from 10 positions) and opt for a significance level of $\alpha = 0.05$. Thus, the critical value of the χ^2 statistic at α is 16,919. The Kruskal–Wallis test returns an H-value of 29,441 and a p-value of 0.000546. Therefore, we can reject the null hypothesis and assume that there is a difference in the distribution of correct solutions across positions.

We further investigate this hypothesis for problems created before and after October 2021. In both cases, we have 9 degrees of freedom and consider a significance level of $\alpha = 0.05$. Thus, the critical value of the χ^2 statistic at α is 16,919. For problems created before October 2021, the Kruskal–Wallis test returns $H = 25{,}971$ and a p-value of 0.0021. Therefore, we can reject the null hypothesis and assume that there is a difference in the distribution of correct solutions across positions when considering problems created before October 2021. When we consider problems created after October 2021, the Kruskal–Wallis test returns an H-value of 6008, which is why we cannot reject the null-hypothesis (the returned H-value leads to a p-value of 0.739, exceeding the significance level). Therefore, we cannot assume that there is a difference in the correctness of solutions per position for problems published after October 2021.

Based on the results of the Kruskal–Wallis test, we know whether there is a difference in the correctness depending on the position or not. However, we do not know how the correctness differs across positions. To further investigate these differences, we tested H2 by comparing the proportions of correct vs. incorrect solutions for each position against all other positions. The results of these 30 χ^2-tests can be found in Table 6. We only observe significant differences in three cases: Considering problems published before October 2021, there is a significant difference in the proportions of correct vs. incorrect solutions when comparing solutions on the first position with the other positions. Further, when considering all problems before and after October 2021, we observe statistically significant differences when comparing the first and the last solution against all others. We further analyzed the statistically significant differences for all problems by comparing the first and the last solution against each other using the χ^2-test. This test reveals significant differences ($\chi^2 = 19.749$, p-value = 8.833e−06). Consequently, the distribution of correct vs. incorrect solutions differs remarkably between Position 1 and Position 10.

We investigate these significant differences in detail by comparing the proportions of correct solutions per position. The results are summarized in Table 5.

There are only few variations in the proportions of correct vs. incorrect solutions for problems generated after October 2021. On average, we find 7.13% of the solutions being correct per position, with

Table 5
Number of correct solutions per position for all problems and split into problems created before October 2021 ("before") and problems created after October 2021 ("after"). Total = the number of problems for which a solution was generated on that position; Corr. = the number of problems that have a correct solution generated on that position; Prob. = the percentage of correct solutions among the generated solutions; all S. = generated solutions on all positions; all P. = problems for which any number of solutions were generated and correct.

	Before			After			All problems		
Pos.	Total	Corr.	Prob.	Total	Corr.	Prob.	Total	Corr.	Prob.
1	1489	533	35.80%	535	29	5.42%	2024	562	27,77%
2	1483	646	43.56%	535	36	6.73%	2018	682	33,80%
3	1472	627	42.60%	530	39	7.36%	2002	666	33,27%
4	1447	612	42.29%	524	39	7.44%	1971	651	33,03%
5	1417	596	42.06%	502	39	7.77%	1919	635	33,09%
6	1355	552	40.74%	468	41	8.76%	1823	593	32,53%
7	1276	528	41.38%	421	32	7.60%	1697	560	33,00%
8	1167	479	41.05%	358	29	8.10%	1525	508	33,31%
9	994	401	40.34%	260	17	6.54%	1254	418	33,33%
10	655	287	43.82%	160	9	5.63%	815	296	36,32%
All S.	12755	5261	41.25%	8093	310	3.83%	17048	5571	32.68%
All P.	1489	1187	79.66%	535	106	19.81%	2024	1293	63,88%

a minimum of 5.42% of correct solutions on position 1 and a maximum of 8.76% of correct solutions on position 6. According to the results of the χ^2-test, these rather small variations are not statistically significant. For problems generated before October 2021, we observe an increased number of correct solutions (both absolutely and relatively) compared to problems created after October 2021 with an average proportion of 41.36% of correct solutions per position, ranging from 35.8% on position 1 to 43.82% on position 10. Given these numbers, we can conclude that when considering problems created before October 2021, the number of correct solutions on position 1 is smaller than on all other positions. According to the results of the χ^2-test, this difference is significant. For all problems, we observe an increased variance in the proportion of correct vs. incorrect solutions per position with an average of 32.94%, a minimum of 27.78% on position 1 and a maximum of 36.32% on position 10. For all other positions, the proportion ranges from 32.53% to 33.80%. These differences between Position 1 and all other positions and position 10 and all other positions are statistically significant according to the results of the χ^2-test.

Overall, we summarize the findings as follows:

Does GitHub Copilot increase the correctness of the first solution through specialized handling? Problems that were published before October 2021 have significantly fewer correct solutions (relatively spoken) on the first position than on every other position.
Problems published after October 2021 share comparable proportions of correct vs. incorrect solutions on each position.
Considering all problems, the first solution has a significantly lower probability of being correct, while a solution on position 10 has a significantly higher probability of being correct.

4.5. RQ3: How many solutions should a developer look at?

The findings above indicate that it is not enough to look at one position only. Of course, the developer might search for correct solutions following a random order. However, we assume for the purpose of this analysis that they search through the solutions starting at position 1, working their way down to 10. This is not unreasonable to assume, as the positioning indicates a priority.

Table 6
Results of Pearson's χ^2 test with Yates' continuity correction.

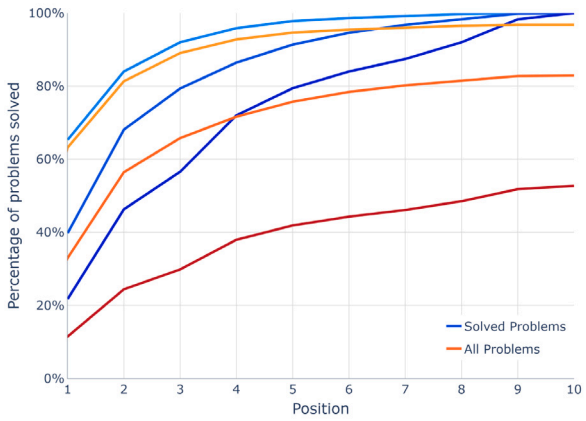
	After		Before		All	
Pos.	χ^2	p-value	χ^2	p-value	χ^2	p-value
1	2.658	0.103	20.414	6.237×10^{-6}	24.931	5.943×10^{-7}
2	0.145	0.703	3.600	0.058	1.242	0.265
3	0.002	0.967	1.187	0.276	0.327	0.567
4	0.014	0.905	0.692	0.406	0.107	0.743
5	0.171	0.680	0.399	0.528	0.146	0.702
6	1.610	0.205	0.139	0.709	0.014	0.906
7	0.048	0.826	0.005	0.943	0.073	0.787
8	0.319	0.572	0.013	0.908	0.274	0.600
9	0.099	0.753	0.325	0.569	0.233	0.6294
10	0.409	0.523	1.772	0.183	4.985	0.026

Difficulty rating. Fig. 8(a) illustrates for that scenario and for problems for which at least one correct solution has been generated the position at which the first correct solution would have been found in our data. Note that the figure differentiates between problems of the three different difficulty ratings. For context, Fig. 8(b) illustrates the same including also the percentage of unsolved problems for each difficulty rating.

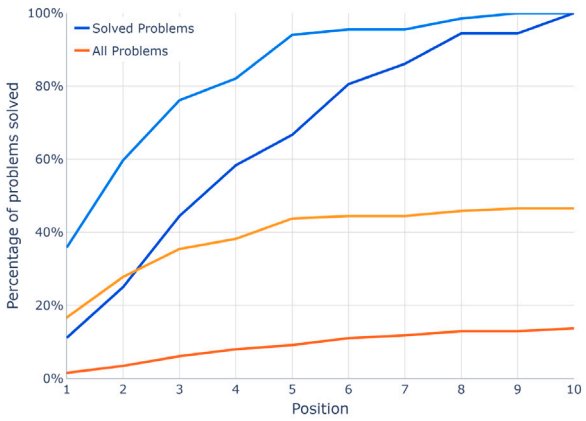
Given the — with difficulty level increasing — likelihood that a problem might not have received a correct solution, a developer might not want to search all generated solutions. Thus, we pose the question of how many solutions a developer would have to look at to find a correct solution for 90% of the problems for which a correct solution has been generated.

As the share of solved and unsolved problems differs with the difficulty rating (see Fig. 7(a)), we look at this question separately for each difficulty rating. Columns 3 in Tables 7, 8 and 9 show for each position the share of problems with a correct solution for which we found a correct solution between the first and the respective position. For example, for 80.23% of the easy problems with a correct solution, we found a correct solution already on position 1 or 2. Our results indicate that for easy problems, one needs to look at up to 4 solutions in order to find a correct solution for at least 90% of the problems for which a correct solution has been generated. For medium problems, one needs to look at up to 5 solutions in order to find a correct solution for at least 90% of the problems for which a correct solution has been generated, and for hard problems, one needs to look at up to 8 solutions.

Given the presence of unsolved problems, the probability of finding a correct solution also decreases the more solutions are looked at without finding a correct solution. A developer might argue that it is not worth continuing to search for a correct solution if the probability



(a) Problems before October 31, 2021.



(b) Problems created after October 31, 2021. We omitted the lines for the "hard" difficulty, as the low sample size of three solved problems is insufficient for any further conclusions regarding the solution position.

Fig. 8. Cumulative line plots for position at which the first correct solution is found sorted (Scenario: searching first at position 1). Darker shades of blue/red implicate a higher difficulty.

Table 7
First correct solution for problems with easy difficulty rating (Scenario: searching first at position 1). 1st correct = share of solved problems for which the first correct solution was found on that position; cumulative 1st correct = share of solved problems for which at least one correct solution was found between position 1 and this position; cum. abs. 1st correct = number of solved problems for which at least one correct solution was found between position 1 and this position; missed 1st correct = number of solved problems for which no correct solution was found between position 1 and this position; prob. find correct = probability to find a correct solution when continuing to search after that position, if so far no correct solution has been found; prob. find no correct = probability to find no correct solution when continuing to search after that position, if so far no correct solution has been found.

	1st correct	Cumulative 1st correct	Cum. abs. 1st correct	Missed 1st correct	Prob. find correct	Prob. find no correct
1	60.70%	60.70%	261	169	65.50%	34.50%
2	19.53%	80.23%	345	85	48.85%	51.15%
3	9.30%	89.53%	385	45	33.59%	66.41%
4	4.19%	93.72%	403	27	23.28%	76.72%
5	3.49%	97.21%	418	12	11.88%	88.12%
6	0.93%	98.14%	422	8	8.25%	91.75%
7	0.47%	98.61%	424	6	6.29%	93.71%
8	0.93%	99.54%	428	2	2.17%	97.83%
9	0.47%	100%	430	0	0%	100%
10	0%	100%	430	0	0%	100%

Table 8
First correct solution for problems with medium difficulty rating (Scenario: searching first at position 1). Legend: see caption of Table 7.

	1st correct	Cumulative 1st correct	Cum. abs. 1st correct	Missed 1st correct	Prob. find correct	Prob. find no correct
1	38.25%	38.25%	262	423	54.02%	45.98%
2	27.59%	65.84%	451	234	39.39%	60.61%
3	11.68%	77.52%	531	154	29.96%	70.04%
4	7.45%	84.97%	582	103	22.24%	77.76%
5	5.11%	90.08%	617	68	15.88%	84.12%
6	3.80%	93.88%	643	42	10.43%	89.57%
7	2.34%	96.22%	659	26	6.71%	93.29%
8	1.90%	98.12%	672	13	3.45%	96.55%
9	1.46%	99.58%	682	3	0.79%	99.21%
10	0.44%	100%	685	0	0%	100%

of finding no correct one exceeds 80%. In that case, the developer might rather try to adjust the used prompt or attempt to solve the problem themselves, e.g. by adjusting one of the previous incorrect solutions.

Columns 7 in Tables 7, 8 and 9 show the probability of finding no correct solution when continuing to search after the given position, if so far no correct solution has been found. We calculate that probability by calculating the share of unsolved problems among the set of problems that are either unsolved or are solved, but where the first correct solution was found after the considered position. For easy and medium problems, the probability of finding no correct solution exceeds 80% after looking at 5 solutions. For hard problems, the probability of finding no correct solution is already higher than 80% after looking at 4 solutions.

Problems created after October 31, 2021. Above we analyzed the question of how many solutions to look at based on difficulty rating. However, as the results so far have shown big differences between problems created before and after October 31, 2021, we will also look at both data sets separately. Note that we do not separate after difficulty rating here, as this would not leave enough data, specifically about hard problems created after October 31, 2021.

For problems created before October 31, 2021, developers would have to look at 5 different solutions to have a 90% chance of finding a correct solution for a problem that has been solved at least once. On

Table 9
First correct solution for problems with hard difficulty rating (Scenario: searching first at position 1). Legend: see caption of Table 7.

	1st correct	Cumulative 1st correct	Cum. abs. 1st correct	Missed 1st correct	Prob. find correct	Prob. find no correct
1	21.91%	21.91%	39	139	33.02%	66.98%
2	24.72%	46.63%	83	95	25.20%	74.80%
3	10.11%	56.74%	101	77	21.45%	78.55%
4	15.17%	71.91%	128	50	15.06%	84.94%
5	7.30%	79.21%	141	37	11.60%	88.40%
6	4.49%	83.70%	149	29	9.33%	90.67%
7	3.37%	87.07%	155	23	7.55%	92.45%
8	4.49%	91.56%	163	15	5.06%	94.94%
9	6.18%	97.74%	174	4	1.41%	98.59%
10	2.25%	100%	178	0	0%	100%

the flip side, after looking at 6 solutions without finding a solution, the probability of finding no more correct solution when continuing to search is 81.67%. In contrast, for problems created after October 2021, developers would have to look at 7 different solutions to have a 90% chance of finding a correct solution for a problem that has been solved. Notably, after looking at only one incorrect solution, the probability of finding no more correct solution when continuing to search is already 87.42%. This analysis is summarized in the tables that are provided with the replication package.

How many solutions should a developer look at? To find a correct solution for at least 90% of the problems for which a correct solution has been generated, one needs to look at up to 4 solutions for easy problems, 5 solutions for medium problems, and 8 solutions for hard problems.
However, after looking at 5 incorrect solutions for easy and medium problems or only 4 incorrect solutions for hard problems, the probability of finding no correct solution among the remaining generated solutions is higher than 80%.
For new problems created after October 2021, to have a 90% probability of finding a correct solution, if a correct solution has been generated, one needs to look at up to 7 different solutions. In addition, the likelihood of not finding a correct solution after no correct solution was found in the first position is 87.42%.

Should a developer skip the first solution?

As described in Section 4.4, we found that in our data the solution at position 1 was less likely to be correct than solutions at position 2 or later. Thus, are there benefits in skipping the first solution when looking for a correct solution?

If a developer wants to look at only one generated solution, one can look at the share of correct solutions among the generated solutions at each position (see Table 5). For problems created before October 31, 2021, this share is between 40.34% and 43.56% for solutions at positions 2 to 10, while it is only 35.8% for solutions generated at position 1. For problems older than October 2021, this share is between 6.54% and 8.76% for solutions at positions 2 to 9, while it is only 5.42% for solutions generated at position 1.

However, what about a scenario where developers skip the first solution and start with the second, working their way down to solution 10, looking at the solution on position 1 only after that? Would developers be able to afford to look at fewer solutions this way, while maintaining the ability to find 90% of the correct solutions?

As shown in Table 10 there is no big difference between the two scenarios. The number of solutions one would have to look at does not change and we found no or only negligible differences in the percentage of solved problems for which a correct solution is found when looking at that many solutions. Similarly, there is no big difference in the number of incorrect solutions one would have to look at until the likelihood of finding no more correct solution crosses 80%.

Table 10
Comparison between two search scenarios, where the developer starts searching for a correct solution at position 1 (scenario 1) or position 2 (scenario 2). More details about the data for scenario 2 can be found in the tables with the replication package. "$prob.cor. > 90\%$" = the number of solutions that need to be looked at to find correct solutions for 90% of the solved problems (and the found percentage of correct solutions if looking at that many solutions in our data); "$prob.not\,sol. > 80\%$" = number of solutions to look at until the probability of finding no correct solutions when continuing to search after that position is > 80% if so far no correct solution has been found (and the found probability in our data).

	Scenario start at 1		Scenario start at 2	
	$prob.cor. > 90\%$	$prob.not\,sol. > 80\%$	$prob.cor. > 90\%$	$prob.not\,sol. > 80\%$
Easy	4 (93.73%)	5 (88.12%)	4 (93.72%)	5 (82.40%)
Medium	5 (90.08%)	5 (84.12%)	5 (90.67%)	5 (84.79%)
Hard	8 (91.56%)	4 (84.94%)	8 (94.93%)	3 (81.50%)
After	7 (91.51%)	1 (87.42%)	7 (92.45%)	1 (88.43%)
Before	5 (91.57%)	6 (81.67%)	5 (91.41%)	6 (81.02%)

Should a developer skip the first solution? If a developer wants to look at one solution only, skipping the first solution will increase the likelihood of finding a correct solution if more than one solution is generated.
However, there is no difference in a scenario where a developer aims to look at enough solutions to find correct solutions for 90% of the solved problems, or tries to avoid looking at additional solutions if the probability of finding no more correct solution becomes higher than 80%.

5. Discussion

Overall, our study points to six key findings.

(1) There is no decrease in the correctness of solutions on later positions, if those are generated. Since most related studies only consider the first solution generated, there seems to be an unconscious expectation that GitHub Copilot presents solutions in a somewhat prioritized order, placing the best solutions at the top. However, our results contradict this idea. Not only did we find fewer correct solutions at position 1 than at other positions, but the number of correct solutions per position was stable (considering cases where solutions on these positions were generated). **Implication for research:** There is potential to improve the accessibility of correct solutions, by introducing an effective prioritization mechanism.

(2) Need to program something that has been programmed frequently? look at 4–5 solutions before adjusting the prompt. For our data set, there is no denying that GitHub Copilot performed best on problems created before October 31, 2021, likely because it has seen those problems and solutions for them in the training data. This ability to generate solutions

for existing problems might be useful in scenarios where developers need source code for standard problems which were included in the training data of GitHub Copilot. In such a scenario, our results indicate that looking at 4–5 solutions provides a 90% or higher chance of finding a correct solution if a correct solution has been generated. After finding 4–5 incorrect solutions the probability of finding no correct solution at all is 80% or higher, especially for hard problems. Having to look at a small set of generated solutions might facilitate developers interacting with GitHub Copilot in acceleration mode. Since these standard problems are likely easier for developers to solve as well, finding a correct solution by pattern matching a mental image of a correct solution could speed up the programming process and decrease the time spent typing out the solution. **Implication for practitioners:** It is worth looking at 4–5 solutions to check whether GitHub Copilot generated a correct one.

(3) For novel or hard problems finding a correct solution should not be the goal of using GitHub Copilot but finding useful starting points. The two performed separations of the problems into problems before and after October 2021 as well as by difficulty rating showed that newer problems as well as hard problems (a) receive fewer correct solutions and (b) require developers to search more generated solutions if they want a good chance of identifying a correct solution. Thus, there is a high effort and low likelihood of finding a correct solution in these cases. However, this does not mean that we do not recommend using GitHub Copilot for hard or novel problems. Instead, we recommend adjusting the expectation of generating a correct solution to the expectation of generating a useful starting point. **Implication for practitioners:** When working with a novel or hard problem it is better to treat GitHub Copilot as a source of inspiration, rather than trying to search for a correct solution.

(4) When using solutions generated by GitHub Copilot as an inspiration, it may be worth skipping the first solution. When using GitHub Copilot as an inspiration, e.g. by adjusting the generated solution manually a developer might want to look at one generated solution, only. The same might be the case, in a scenario where a developer is doing an iterative prompt improvement and inspects one generated solution in order to know how to adjust the prompt. Our results indicate that the first position is not the best position to look at as it has a lower share of correct solutions. Independent of the mode that a developer is currently in when interacting with GitHub Copilot — whether in acceleration or exploration mode — our findings suggests that if you are only looking at one solution, you should look at the second. Note that this recommendation also invalidates the supposed benefit of GitHub Copilot's multi-suggestion feature to some degree. However, the autocomplete feature of GitHub Copilot usually generates much smaller portions of code compared to the multi-suggestion feature. Therefore, it is mostly necessary to use this multi-suggestion feature to generate larger chunks of code which developers in exploration mode might be looking for. **Implication for practitioners:** If you plan to look at one solution only, skip the first solution. **Implication for tool developers:** Generating more extensive blocks of code, while not wanting to wait for the tool to generate up to 10 of them should be something to consider in order to support developers when iterating over a prompt. Additionally, the reason for a lower percentage of correct solutions on the first position should be revisited by the tool developers to mitigate this potential drawback.

(5) The effort of looking through all generated solutions might not be justified. Our recommendation not to search all generated solutions is based on the assumption that developers require a period of time to assess whether a generated solution is correct or not. However, we know that the time it takes to assess the correctness of a solution will vary. For example, there are generated solutions that suffer from more obvious syntax errors while other solutions will be wrong on a logic level. Thus, it is unclear at what point it becomes easier to adjust the prompt or directly adjust a generated solution than to continue to search through multiple generated solutions for the same prompt. For the purpose of this paper, we set the thresholds to a 90% probability to find a correct solution, if one was generated and an 80% probability that no more correct solution can be found. However, while meaningful, it is possible that other thresholds are more valid if considering the alternatives of adjusting prompts and solutions. **Implication for research:** We need data comparing the effort to understand and judge multiple generated solutions with the effort of readjusting the prompt and checking the results iteratively or even adjusting an imperfect generated solution. **Implication for tool developers:** As it is often not worth it to inspect all solutions, it is also often not worth it to generate 10 solutions, as this takes additional time as well. We therefore suggest reexamining and potentially decrease the amount of solutions which should be generated at a time.

(6) Our results invalidate the expectation that examining more solutions leads to a higher proportion of solved problems, as they are similar to related studies that consider only one generated solution. Related studies, such as the study by Liu et al. [5] and others [33–35] generate only one solution per coding problem (and per programming language) and maybe iterate over the prompt. As described in Section 4.2 we initially considered our results to be consistent with these previous studies. As we generated and evaluated up to 10 solutions, we expected an increased number of solved problems, which we found (from 50% solved problems in Python found by Liu et al. to 63.9% solved problems in our study). However, we found that by looking at a single position only, even in the best case one would miss finding a correct solution for up to 50% of the problems for which a correct solution was generated. Given this finding, we should have seen a larger difference between our and Liu et al.'s numbers. Nonetheless, we find correctness ratings that are too similar to the ones of Liu et al. who report that 75.35% of problems created before 2022 were solved (we found an increase in solved problems of 4.25pp for problems before October 2021) and a 23.93% acceptance rate for problems after 2021 (we found a decrease of 4.13pp for problems after October 2021). Note that these are the numbers that Liu et al. measured before iterating over the prompts. **Implication for research:** We can only speculate why we do not find higher numbers of correctness, given that we looked at more generated solutions. The difference in the cut-off date (Liu et al. used January 2022, while we used October 2021), would rather lead to Liu et al. underestimating the reported acceptance rate for problems before 2022. Thus, this cannot explain why our results are not much better than theirs. Another possible explanation is the sampling. We attempted to include — as far as possible — a complete set of LeetCode tasks in our study, working with 2025 problems. Liu et al. used a subset of 728 problems. However, they report a similar distribution of easy-medium-hard problems as we have (roughly 1:2:1). Consequently, future research is required to explain this mismatch. **Implications for tool developers:** Again, as the multitude of solutions does not seem to provide the desired value, we suggest reexamining and potentially lowering the amount of generated solutions.

6. Threats to validity

The results of our study are subject to threats to validity which we discuss in the following. According to Wohlin et al. [47], we distinguish between threats to *construct, internal, conclusion*, and *external validity*.

Construct validity. Our study is to some extent comparable to an uncontrolled experiment, in which we are unable to control several core parameters, such as the performance of human developers (influencing the acceptance rate) or the validity of the provided metadata (including the difficulty rating which we cannot prove being objective). However, we tried to mitigate these facts by using all available coding problems (i.e., accessible without premium accounts) that did not satisfy any of our exclusion criteria. Overall, we analyzed more than 17 000 solutions, which is why we achieved a diverse data base used for the analysis.

Internal validity. In the study, we wanted to measure the correctness of GitHub Copilot's solutions on coding problems in general. However, caused by our exclusion criteria, we limit the analysis to coding problems that are not affected by the exclusion criteria. This influences the results, as excluded problems might have been less likely to be solved. In particular, we did not study specific types of coding problems, such as problems with complex mathematical symbols or problems with pictures that are essential for understanding. It is likely that for such problems GitHub Copilot shows different (potentially lower) success rates than for the problems we studied.

Moreover, we used GitHub Copilot without the context of a real code base by only using minimal prepared files to include the information needed for a particular coding problem. In a real setting, GitHub Copilot would be able to make use of the context surrounding a problem and be able to reuse code of similar problems that are already solved. We therefore might underestimate GitHub Copilot's ability to generate correct solutions by effectively removing this context in our study.

Another issues arises for the determined cut-off date of October 31, 2021. As we cannot be certain, that problems created before this date have been part of the training data and problems created afterwards have not been, or vice versa. This risk is further increased as our estimate of creation date is likely worse for older problems due to a lower popularity of LeetCode in the past. We do not expect this to drastically alter our results, however, we might overestimate or underestimate GitHub Copilot's ability to solve known or unknown problems. As we use those results to make implications about how to approach the generated solutions, this might influence the quantitative aspects of those implications. Still, we argue that the general notions presented in the discussion persist even if quantitative aspects vary.

A further important issue is our inclusion of coding problems that have likely been part of the training data of GitHub Copilot. This could have a strong impact on the results of the study and our recommendations. However, general tendencies, such as a decrease of correctness with increasing difficulty and decreasing acceptance rate, have been shown to be similar (cf. Fig. 7). This similarity also applies to the trends regarding the positions with correct solutions as shown in Fig. 8. The percentage of correct solutions of early positions is lower for data after October 31, 2021. Still, there is something that we can learn from the data before that date, which is a confirmation that the correctness of easy, medium, and hard problems differs from one another as expected, with hard problems showing the lowest number of correct solutions for early positions. As we do not have enough data for solved hard problems after the cut-off date, the observation from the data before the cut-off for hard problems together with the data on easy and medium problems created after the cut-off gives us at least some insights into what to expect regarding such tasks. We further argue that interpreting the results for problems created before October 31, 2021 provides value to developers. Since GitHub Copilot was trained on a large amount of data, it is likely that many common problems and their solutions that are independent of new practices and technologies were also seen during training. Therefore providing metrics to those problems and interpreting them can prove useful to developers as well.

However, this also raises the issue that even if a coding problem and its description was included in the training data of GitHub Copilot, it is unclear whether the use of similar descriptions would yield the same results. In a real setting, a developer is unlikely to use the same description to a coding problem as LeetCode did. That means even though a description is meant to describe the same "standard" problem, changes in the wording might lead to a worse performance of GitHub Copilot. This effect of paraphrasing descriptions — keeping semantics similar, but changing them syntactically — has been researched by Mastropaolo et al. [48]. Their results suggest that while paraphrased descriptions seem to change the generated code and even whether it solves the corresponding problem or not, the percentage of solved problems is relatively constant. With this, we argue that our results would be similar for paraphrased descriptions as well.

Conclusion validity. Our results emerge from a large data set with 2025 coding problems and more than 17 000 solutions. While some of our results are based on descriptive statistics only, others are shown by hypotheses testing with a significance level of $\alpha = 0.05$. This way, we were able to gain objective results that are not subject to any interpretation. In most cases, we obtain a very small p-value, indicating strong and reliable results. However, future studies are required to strengthen our results.

External validity. Our results are limited in their generalizability to other contexts. We cannot make any statements for other programming languages such as JavaScript and C. Moreover, the coding problems we used may not be representative of tasks that developers have to face. In contrast to the isolated problems that LeetCode provides, real tasks may include interacting with multiple different APIs while using specific hardware, frameworks, and libraries. Thus, we cannot make any statements about the interaction with different software and hardware systems.

Furthermore, by using descriptions, some of which exceeded 1000 characters, and method signatures, we provide a mass of information to GitHub Copilot for each coding problem that might be unrealistic in a real setting. Depending on the problem and the description, this could be beneficial or disadvantageous for generating correct code, as it could provide essential information or weaken the attention to important details of the description.

By focusing on the correctness of solutions only, our recommendations are not easily generalized to the context of the exploration mode, in which developers aim to find starting points. However, while complete correctness might not be a developer's initial goal when generating code with GitHub Copilot, the ultimate goal is usually to have correct code. We argue that our recommendations should not negatively impact the search for good starting points while maximizing the chance of already finding a correct solution.

7. Conclusions

In this paper, we conducted a study to replicate results concerning GitHub Copilot's ability to generate correct solutions for coding problems. As a further contribution, we investigated how a developer should inspect the multitude of generated solutions for a given coding problem. Using our results for the research questions, we answer the question of how a developer should inspect the multitude of generated solutions as follows: The number of solutions and the way of prioritizing positions varies according to the type of problem and the individual goal of the developer. As a first rule, there is no decrease in the average correctness of solutions at later positions, meaning that correct solutions are not more likely to occur at the beginning. Problems perceived as easy or those that are well-known tend to be answered correctly by the first 4–5 solutions. This suggestion also questions the design of GitHub Copilot, as generating up 10 solutions adds little value while simultaneously increasing the time it takes to generate solutions.

Hard problems or those that GitHub Copilot has likely not encountered during training, are less likely solved and looking for correct solutions is thus often not effective or efficient. We base this recommendation on our results in Section 4.5, which show a strong individual effect of the creation date and difficulty on the chance of finding a correct solution.

To maximize efficiency, e.g. when looking at one solution only, our data further suggests that skipping the first presented solution and looking at the second solution instead is beneficial to find correct solutions. We argue that this finding is related to the significance of the first solution and therefore hypothesize that GitHub Copilot employs special handling, for example by altering the prompt. We do not believe that GitHub Copilot aims to decrease the correctness of the first solution. However, this may be a side effect of an effort to generate solutions that are, e.g., more readable or contain more comments. If

this was true, this also implies that tool developers should potentially reevaluate their tool configuration.

For the interpretation of our results and suggestions, we want to point out, that they are limited in their generalizability to other contexts as we used isolated programming problems which are not influenced by frameworks or libraries outside of the direct control of a developer. Moreover, lacking the project context may also negatively impact the performance of GitHub Copilot and potentially alter the results.

In future work, we intend to validate our claims in the discussion through further developer studies. In particular, we will investigate the effort required by developers to check generated solutions and to adjust the prompt or the generated solution in order to refine our recommendations for working with AI-powered tools in software development.

CRediT authorship contribution statement

Julian Oertel: Writing – review & editing, Writing – original draft, Visualization, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Jil Klünder:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Investigation, Formal analysis, Conceptualization. **Regina Hebig:** Writing – review & editing, Writing – original draft, Validation, Supervision, Project administration, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

We have uploaded a replication package to Zenodo and include a reference in the paper.

References

[1] C. Ebert, P. Louridas, Generative AI for software practitioners, IEEE Softw. 40 (4) (2023) 30–38.

[2] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, J.M. Zhang, Large language models for software engineering: Survey and open problems, in: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering, ICSE-FoSE, 2023, pp. 31–53, http://dx.doi.org/10.1109/ICSE-FoSE59343.2023.00008.

[3] J. Wang, Y. Chen, A review on code generation with LLMs: Application and evaluation, in: 2023 IEEE International Conference on Medical Artificial Intelligence, MedAI, 2023, pp. 284–289, http://dx.doi.org/10.1109/MedAI59581.2023.00044.

[4] S. Barke, M.B. James, N. Polikarpova, Grounded copilot: How programmers interact with code-generating models, 2022, arXiv:2206.15000. URL https://arxiv.org/abs/2206.15000.

[5] Z. Liu, Y. Tang, X. Luo, Y. Zhou, L.F. Zhang, No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT, IEEE Trans. Softw. Eng. 50 (6) (2024) 1548–1584, http://dx.doi.org/10.1109/TSE.2024.3392499.

[6] Anonymous, No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT - artifact, 2024, http://dx.doi.org/10.5281/zenodo.10556350.

[7] Y. Tian, T. Zhang, Selective prompt anchoring for code generation, 2024, arXiv:2408.09121. URL https://arxiv.org/abs/2408.09121.

[8] LeetCode, LeetCode, 2023, Online. URL https://leetcode.com/.

[9] M.-F. Wong, S. Guo, C.-N. Hang, S.-W. Ho, C.-W. Tan, Natural language generation and understanding of big code for AI-assisted programming: A review, Entropy 25 (6) (2023) http://dx.doi.org/10.3390/e25060888, URL https://www.mdpi.com/1099-4300/25/6/888.

[10] A.E. Howe, A.v. Mayrhauser, R.T. Mraz, Test case generation as an AI planning problem, Autom. Softw. Eng. 4 (1) (1997) 77–106, http://dx.doi.org/10.1023/A:1008607721339.

[11] S. Proksch, J. Lerch, M. Mezini, Intelligent code completion with Bayesian networks, ACM Trans. Softw. Eng. Methodol. 25 (1) (2015) http://dx.doi.org/10.1145/2744200.

[12] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, et al., Improving language understanding by generative pre-training, 2018.

[13] M. Chen, J. Tworek, H. Jun, Q. Yuan, H.P.d. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F.P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W.H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A.N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, W. Zaremba, Evaluating large language models trained on code, 2021, arXiv:2107.03374.

[14] N. Friedman, Introducing GitHub copilot: Your AI pair programmer, 2021, Online. URL https://github.blog/2021-06-29-introducing-github-copilot-ai-pair-programmer/.

[15] T. Dohmke, GitHub copilot is generally available to all developers, 2022, Online. URL https://github.blog/2022-06-21-github-copilot-is-generally-available-to-all-developers/.

[16] Z.C. Ani, Z.A. Hamid, N.N. Zhamri, The recent trends of research on GitHub copilot: A systematic review, in: N.H. Zakaria, N.S. Mansor, H. Husni, F. Mohammed (Eds.), Computing and Informatics, Springer Nature Singapore, Singapore, 2024, pp. 355–366.

[17] Z.K. Cui, M. Demirer, S. Jaffe, L. Musolff, S. Peng, T. Salz, The effects of generative AI on high skilled work: Evidence from three field experiments with software developers, 2024, http://dx.doi.org/10.2139/ssrn.4945566, Available At SSRN 4945566.

[18] P. Vaithilingam, T. Zhang, E.L. Glassman, Expectation vs. Experience: Evaluating the usability of code generation tools powered by large language models, in: Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems, CHI EA '22, Association for Computing Machinery, New York, NY, USA, 2022, http://dx.doi.org/10.1145/3491101.3519665.

[19] S. Peng, E. Kalliamvakou, P. Cihon, M. Demirer, The impact of AI on developer productivity: Evidence from GitHub copilot, 2023, arXiv:2302.06590.

[20] S. Imai, Is GitHub copilot a substitute for human pair-programming? An empirical study, in: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, ICSE '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 319–321, http://dx.doi.org/10.1145/3510454.3522684.

[21] A. Ziegler, E. Kalliamvakou, S. Simister, G. Sittampalam, A. Li, A. Rice, D. Rifkin, E. Aftandilian, Productivity assessment of neural code completion, 2022, arXiv:2205.06537.

[22] A. Moradi Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M.C. Desmarais, Z.M.J. Jiang, GitHub copilot AI pair programmer: Asset or liability? J. Syst. Softw. 203 (2023) 111734, http://dx.doi.org/10.1016/j.jss.2023.111734, URL https://www.sciencedirect.com/science/article/pii/S0164121223001292.

[23] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, R. Karri, Asleep at the keyboard? Assessing the security of GitHub copilot's code contributions, in: 2022 IEEE Symposium on Security and Privacy, SP, 2022, pp. 754–768, http://dx.doi.org/10.1109/SP46214.2022.9833571.

[24] O. Asare, M. Nagappan, N. Asokan, A user-centered security evaluation of copilot, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, ICSE '24, Association for Computing Machinery, New York, NY, USA, 2024, http://dx.doi.org/10.1145/3597503.3639154.

[25] O. Asare, M. Nagappan, N. Asokan, Is GitHub's copilot as bad as humans at introducing vulnerabilities in code? Empir. Softw. Eng. 28 (6) (2023) 129, http://dx.doi.org/10.1007/s10664-023-10380-1.

[26] Y. Fu, P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, J. Chen, Security weaknesses of copilot generated code in GitHub, 2024, arXiv:2310.02059. URL https://arxiv.org/abs/2310.02059.

[27] V. Majdinasab, M.J. Bishop, S. Rasheed, A. Moradidakhel, A. Tahir, F. Khomh, Assessing the security of GitHub copilot's generated code — A targeted replication study, in: 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER, 2024, pp. 435–444, http://dx.doi.org/10.1109/SANER60148.2024.00051.

[28] J. Res, I. Homoliak, M. Perešíni, A. Smrčka, K. Malinka, P. Hanacek, Enhancing security of AI-based code synthesis with GitHub copilot via cheap and efficient prompt-engineering, 2024, arXiv:2403.12671. URL https://arxiv.org/abs/2403.12671.

[29] M. Wermelinger, Using GitHub copilot to solve simple programming problems, in: Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1, SIGCSE 2023, Association for Computing Machinery, New York, NY, USA, 2023, pp. 172–178, http://dx.doi.org/10.1145/3545945.3569830.

[30] S. Lau, P. Guo, From "ban it till we understand it" to "resistance is futile": How university programming instructors plan to adapt as more students use AI code generation and explanation tools such as ChatGPT and GitHub copilot, in: Proceedings of the 2023 ACM Conference on International Computing Education Research, vol. 1, ICER '23, Association for Computing Machinery, New York, NY, USA, 2023, pp. 106–121, http://dx.doi.org/10.1145/3568813.3600138.

[31] F. Nizamudeen, L. Gatti, N. Bouali, F. Ahmed, Investigating the impact of code generation tools (ChatGPT & Github CoPilot) on programming education.
[32] B. Puryear, G. Sprint, Github copilot in the classroom: Learning to code with AI assistance, J. Comput. Sci. Coll. 38 (1) (2022) 37–47.
[33] B. Yetiştiren, I. Özsoy, M. Ayerdem, E. Tüzün, Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub copilot, Amazon CodeWhisperer, and ChatGPT, 2023, arXiv:2304.10778.
[34] B. Yetistiren, I. Ozsoy, E. Tuzun, Assessing the quality of GitHub copilot's code generation, in: Proceedings of the 18th International Conference on Predictive Models and Data Analytics in Software Engineering, PROMISE 2022, Association for Computing Machinery, New York, NY, USA, 2022, pp. 62–71, http://dx.doi.org/10.1145/3558489.3559072.
[35] N. Nguyen, S. Nadi, An empirical evaluation of GitHub copilot's code suggestions, in: Proceedings of the 19th International Conference on Mining Software Repositories, MSR '22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 1–5, http://dx.doi.org/10.1145/3524842.3528470.
[36] V. Corso, L. Mariani, D. Micucci, O. Riganelli, Generating Java methods: An empirical assessment of four AI-based code assistants, in: Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension, ICPC '24, Association for Computing Machinery, New York, NY, USA, 2024, pp. 13–23, http://dx.doi.org/10.1145/3643916.3644402.
[37] X. Zhou, P. Liang, B. Zhang, Z. Li, A. Ahmad, M. Shahin, M. Waseem, Exploring the problems, their causes and solutions of AI pair programming: A study on GitHub and stack overflow, 2024, arXiv:2311.01020. URL https://arxiv.org/abs/2311.01020.
[38] B. Zhang, P. Liang, X. Zhou, A. Ahmad, M. Waseem, Demystifying practices, challenges and expected features of using GitHub copilot, 2023, arXiv:2309.05687.
[39] B. Zhang, P. Liang, X. Zhou, A. Ahmad, M. Waseem, Practices and challenges of using GitHub copilot: An empirical study, in: Proceedings of the 35th International Conference on Software Engineering and Knowledge Engineering, SEKE2023, KSI Research Inc., 2023, http://dx.doi.org/10.18293/seke2023-077.
[40] J. Sauvola, S. Tarkoma, M. Klemettinen, J. Riekki, D. Doermann, Future of software development with generative AI, Autom. Softw. Eng. 31 (1) (2024) 26, http://dx.doi.org/10.1007/s10515-024-00426-z.
[41] Microsoft, Playwright, 2023, Online. URL https://playwright.dev/.
[42] P. Waritschlager, AHK_X11, 2023, Online. URL https://github.com/phil294/AHK_X11.
[43] J. Oertel, J. Klünder, R. Hebig, Replication package for "don't settle for the first! how many GitHub copilot solutions should you check?", 2024, http://dx.doi.org/10.5281/zenodo.13760371.
[44] W.H. Kruskal, W.A. Wallis, Use of ranks in one-criterion variance analysis, J. Amer. Statist. Assoc. 47 (260) (1952) 583–621, http://dx.doi.org/10.1080/01621459.1952.10483441, arXiv:https://www.tandfonline.com/doi/pdf/10.1080/01621459.1952.10483441.
[45] K. Pearson, X. On the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling, Lond. Edinb. Dublin Philos. Mag. J. Sci. 50 (302) (1900) 157–175, http://dx.doi.org/10.1080/14786440009463897, arXiv:https://doi.org/10.1080/14786440009463897.
[46] F. Yates, Contingency tables involving small numbers and the χ 2 test, Suppl. J. R. Stat. Soc. 1 (2) (1934) 217–235.
[47] C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, Springer Science & Business Media, 2012.
[48] A. Mastropaolo, L. Pascarella, E. Guglielmi, M. Ciniselli, S. Scalabrino, R. Oliveto, G. Bavota, On the robustness of code generation techniques: An empirical study on GitHub copilot, in: 2023 IEEE/ACM 45th International Conference on Software Engineering, ICSE, 2023, pp. 2149–2160, http://dx.doi.org/10.1109/ICSE48619.2023.00181.